

rjk::duck - Beautiful Type Erasure with C++26 Reflection

by Ryan Keane

Zero boilerplate runtime dispatch.

rjk::duck uses reflection, define_aggregate, and inheritance / pointer-interconvertibility to give you the tools to build simple, composable interfaces.

```
struct Container {
    auto size() const -> std::size_t;
    auto empty() const -> bool;
    auto clear() -> void;
};

rjk::duck<Container> d{std::vector{1, 2, 3}};
d.size(); // 3

d = std::string{"hello"}; // swap type at runtime
d.size(); // 5

d = std::map<int, int>{{1, 2}, {3, 4}};
d.empty(); // false
d.clear();
d.empty(); // true
```

Basic usage of duck.

```
template <typename Ret, typename... Args>
struct Trait<Ret(Args...)> : rjk::copyable {
    [[rjk::direct]]
    Ret operator()(Args...) const;
};

template <typename Func>
using function = rjk::duck<Trait<Func>>;

template <typename Func>
using function_ref = rjk::duck_view<Trait<Func>>;
```

std::function and function_ref implemented with duck.

Virtual Functions

- <10 lines
- Intrusive
- Pointer semantics

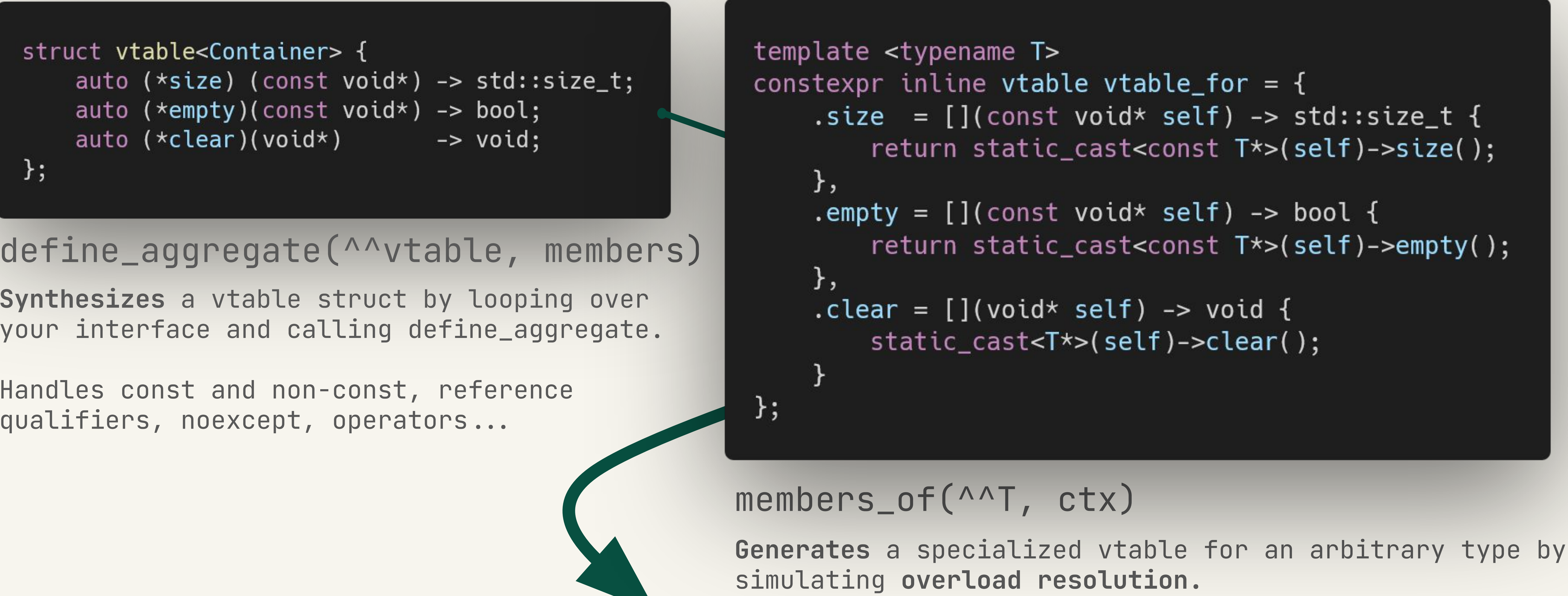
Manual Type Erasure

- 100+ lines
- Non-intrusive
- Value semantics

rjk::duck

- <10 lines
- Non-intrusive
- Value semantics

Implementation Pipeline



```
struct function_wrapper<"size"> {
    [[no_unique_address]]
    vtable_function<&vtable::size> size;
};

struct function_wrapper<"clear"> {
    [[no_unique_address]]
    vtable_function<&vtable::clear> clear;
};

struct function_wrapper<"empty"> {
    [[no_unique_address]]
    vtable_function<&vtable::empty> empty;
};
```

```
duck inherits from one base class for each interface function. Each stores an empty callable with the same name as the interface member.

class duck_view<Container> : public function_wrappers_for<Container> {
private:
    void* object;
    const vtable<Container>* vtable;

public:
    template <typename T>
    duck_view(const T& obj) : object(&obj), vtable(&vtable_for<T>) {}
};
```

```
Ret vtable_function::operator()(Args... args) {
    auto* enclosing_ptr = reinterpret_cast<function_wrapper*>(this);
    auto* duck_ptr = static_cast<duck_view*>(enclosing_ptr);

    return duck_ptr->vtable->[vtable_member:]{
        duck_ptr->obj, std::forward<Args>(args)...;
    }
}
```

Pointer interconvertibility lets the wrapper function cast down the inheritance chain to duck, then dispatch to the vtable.

What's next?

C++26

- Default implementations
- variant-style backend
- Customizable overload resolution

C++29 and beyond

- Token injection?
- constexpr?
- P4148: Structural Subtyping for C++

Additional Features

- Owning and view semantics
- Operator support
- Covariant return types
- Multi-trait composition
- Narrowing from larger trait sets to smaller ones
- Extension options for existing types
- SBO size customization
- Allocator support
- ...and more!

Benchmarks

CONSTRUCTION & DESTRUCTION					
Bytes	rjk::duck	std::unique_ptr	std::function	AnyAny	Proxy
Construction					
16	0.874 ns	8.49 ns	0.774 ns	0.865 ns	0.769 ns
32	9.09 ns	8.63 ns	8.86 ns	0.836 ns	8.64 ns
64	8.58 ns	8.60 ns	8.75 ns	8.57 ns	8.74 ns
Destruction					
16	1.41 ns	6.59 ns	2.03 ns	1.84 ns	1.85 ns
32	6.49 ns	6.17 ns	6.96 ns	1.72 ns	6.46 ns
64	6.63 ns	6.24 ns	7.20 ns	7.28 ns	6.78 ns

```
template <bool Direct>
struct Trait {
    [[rjk::direct(Direct)]]
    void foo() const;
};
```

Mark a member function as direct to remove an indirection.

```
; Two loads
test_indirect(rjk::duck<Trait<false>> const&):
    ldp    x0, x1, [x0]
    ldr    x1, [x1, #40]
    mov    x16, x1
    br     x16

; Two loads
test_virtual(IITrait const&):
    ldr    x1, [x0]
    ldr    x1, [x1]
    mov    x16, x1
    br     x16

; One load
test_direct(rjk::duck<Trait<true>> const&):
    ldp    x0, x1, [x0]
    mov    x16, x1
    br     x16
```



Try it now on Compiler Explorer!

Project Requirements:

- gcc-16 or later
- -std=c++26
- -freflection

Contact: keaner@rpi.edu



GitHub



Blog Post